

adder-viz: Real-Time Visualization Software for Transcoding Event Video

Andrew C. Freeman
andrew_freeman@baylor.edu
Baylor University
Waco, Texas, USA

Luke Reinkensmeyer
luke_reinkensmeyer1@baylor.edu
Baylor University
Waco, Texas, USA

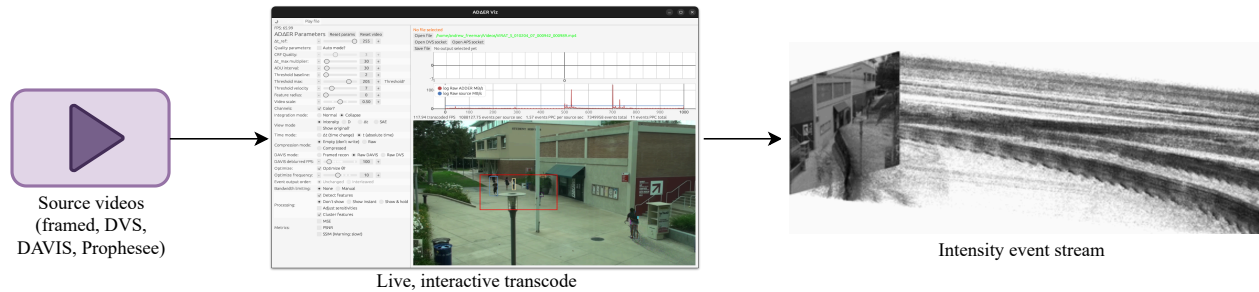


Figure 1: Usage of the *adder-viz* interface. The small bounding boxes are obtained through FAST feature clustering. The red bounding box is a manually-drawn ROI. Regions outside this ROI (e.g., the cyclist) have lower sensitivity. These pixels produce few events, and experience temporal averaging of their intensities.

Abstract

Recent years have brought about a surge in neuromorphic “event” video research, primarily targeting computer vision applications. Event video eschews video frames in favor of asynchronous, per-pixel intensity samples. While much work has focused on a handful of representations for specific event cameras, these representations have shown limitations in flexibility, speed, and compressibility. We previously proposed the unified ADAER representation to address these concerns. This paper introduces numerous improvements to the *adder-viz* software for visualizing real-time event transcode processes and applications in-the-loop. The MIT-licensed software is available from a centralized repository at <https://github.com/ac-freeman/adder-codec-rs>.

CCS Concepts

• **Computing methodologies** → Image compression; **Image representations; Image processing**; • **Human-centered computing** → *Visualization systems and tools*.

Keywords

visualization, event representation, event video, video processing, event vision

ACM Reference Format:

Andrew C. Freeman and Luke Reinkensmeyer. 2025. *adder-viz: Real-Time Visualization Software for Transcoding Event Video*. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*, October 27–31, 2025, Dublin, Ireland. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3746027.3756867>

1 Introduction

Traditionally, video is structured as a sequence of discrete image frames. Recently, however, a novel video sensing paradigm has emerged which eschews video frames entirely. These “event” sensors aim to mimic the human vision system with asynchronous sensing, where each pixel has an independent, sparse data stream. While these cameras enable high-speed and high-dynamic-range sensing, researchers often revert to a framed representation of the event data for existing applications, or build bespoke applications for a particular camera’s event data type. At the same time, classical video systems have significant computational redundancy at the application layer, because pixel samples are repeated across frames in the uncompressed domain.

To address the shortcomings of existing systems, we previously introduced Address, Decimation, Δt Event Representation (ADAER, pronounced “adder”), a novel intermediate video representation and system framework [10, 11]. The framework transcodes a variety of framed and event camera sources into a single event-based representation. This unified representation supports source-modeled lossy compression and backward compatibility with traditional frame-based applications.

Since the event video paradigm represents a dramatic shift from the mechanisms of classical video, we seek a simple, user-friendly program to increase the accessibility and reproducibility of this research avenue. Our *adder-viz* software aims to fill this gap. While we



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

MM '25, Dublin, Ireland

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2035-2/2025/10

<https://doi.org/10.1145/3746027.3756867>

previously introduced this software as a component of the broader ADAER framework, it was severely limited in performance, reliability, accuracy, and features. In this work, we discuss the overall ADAER framework, then introduce our recent advancements to our open-source software packages, including:

- Improve the accuracy and clarity of transcode operations
- Remove dependency on unstable Rust features
- Expand interactive rate control applications with feature clustering and manual ROIs
- Add helper overlays for each transcoder setting
- Reduce the compiled binary size by 75%
- Increase live compression speed by up to 275%

2 Background

2.1 Framed Video

Since a traditional video is simply a sequence of discrete images, we see that an uncompressed video frame representation has high temporal and spatial redundancy. This redundancy can produce enormous data rates, especially at high frame rates and resolutions. Since the early days of video systems, researchers have sought to mitigate these rate concerns through compression, so that the bandwidth and storage capabilities of consumer devices can receive and process video with reasonable speed. The compression and application layers, however, are largely divorced in classical video systems, and compression performance does not correlate directly with application speed. This quality is most severe in systems where the video does achieve high temporal compression, such as surveillance and high-speed video. An application may incorporate a differencing mechanism to remove temporal redundancy, but such a scheme does not scale with changes to the input frame rate.

2.2 Event Video

Event video, on the other hand, detaches intensity samples from a strictly synchronous representation. Neuromorphic event cameras have recently received attention in computer vision research because they have microsecond temporal resolution, consume little power, and extremely high dynamic range (>120 dB) [12]. The fundamental advancement that makes these features possible is an asynchronous sensor circuit: whereas traditional sensors record the integrated value for all pixels at the same time, event-based sensors record an individual pixel's value only when it has met a certain threshold criterion.

The most common event camera leverages the Dynamic Vision System (DVS) [12, 14]. A DVS pixel records (or “fires”) a tuple $\langle x, y, p, t \rangle$ at timestamp t when it experiences a change in instantaneous log intensity that exceeds its global threshold, θ [12]. As a consequence of this change-based sensing, these cameras predominantly fire events that correspond to moving edges between areas of high contrast. Since static regions are thus not captured directly, researchers have sought to utilize multimodal sensing with DVS and frame-based cameras. The Dynamic and Active Vision System (DAVIS) combines a DVS and framed sensor on a single sensor package [3, 4]. The camera outputs separate data streams for frames and events, however, leaving sensor fusion as a downstream task.

Various event camera manufacturers have their own proprietary data formats. Furthermore, many event-based applications are designed around a number of frame-based representations of the event data [2, 13, 15, 16, 19]. This process inevitably quantizes the event timestamps, and it leaves any rate control mechanism up to a traditional frame-based encoder. Typical vision applications are designed around a single event format and representation, which makes real-world system integration difficult.

2.3 ADAER

Our ADAER system addresses some major weakness of both traditional framed video and existing event video representations. Chiefly, it transcodes video from a variety of sources into a **single event representation**. This transcode process supports variable loss of the raw representation, application in-the-loop support, and source-modeled compression.

In the transcoding process, an ADAER event tuple is fired when a pixel's change in intensity accumulates to an amount past a certain threshold value. An event tuple $\langle x, y, c, D, t \rangle$ stores the spatial coordinates (x) and (y), the color channel (c), the intensity threshold (D), and the timestamp (t). The threshold D is determined by both the current pixel brightness and the time elapsed since the last event was fired [11]. D can be tuned further based on application-level demands, such as regional saliency [10]. The intensity of a pixel is calculated by the equation $I = \frac{2^D}{\Delta t}$, where Δt is the time elapsed since the pixel last fired an event. Unlike DVS events, which merely represent intensity *change*, ADAER events directly convey *absolute intensity*.

Key advantages to ADAER include the following: it is source agnostic, so downstream applications may be transparently applied to both framed- and event-based video sources, including upcoming event cameras; it is trivial to produce a framed representation from ADAER, so it maintains compatibility with a plethora of framed applications; it can dramatically reduce the raw (uncompressed) data rate of videos with high temporal redundancy, making vision applications faster [10]; and, finally, it provides clear methods for introducing dynamic temporal loss for rate adaptation.

An example scenario where ADAER can excel is computer vision analysis for surveillance systems. Other potential application areas include streaming systems for robotics, autonomous vehicles, and mixed reality. On a framed surveillance dataset, we found that ADAER transcoding can reduce the amount of raw data by more than 90% [8]. If vision applications are adapted to operate on this temporally sparse representation, they can be made significantly faster. For example, we saw up to a 43% speed-up in FAST feature detection in [10]. If ADAER transcoding occurs on-camera at the edge, then we can see faster transmission, reduced storage costs, and lower latency analysis. Furthermore, non-framed sensors such as a DAVIS camera may be integrated without requiring a separate processing pipeline.

3 Software Overview

We first introduced the open-source ADAER software suite in [9]. It consists of several libraries and applications written in Rust. We provide a modular interface for reading and transcoding data from multiple event cameras and framed video formats, to produce a

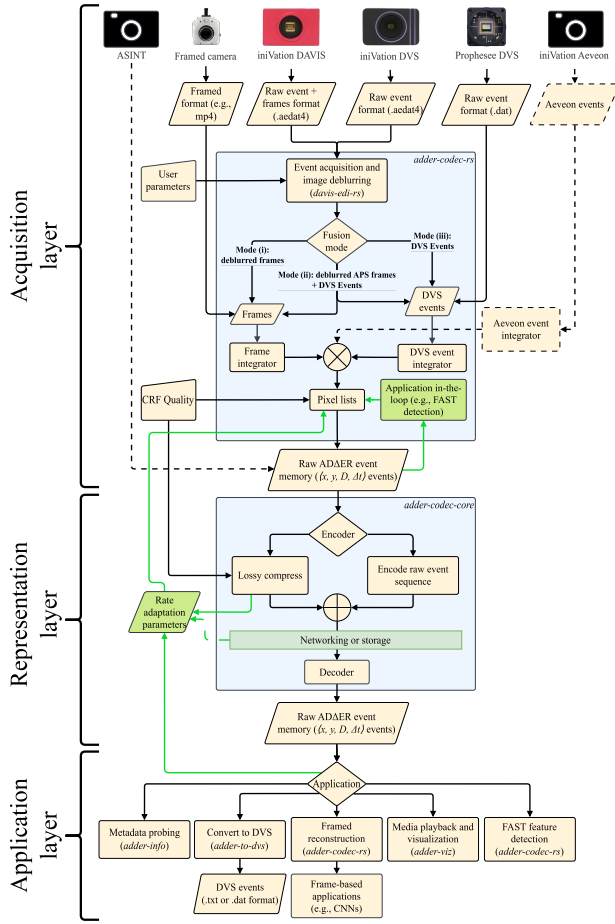


Figure 2: Detailed diagram of the three-layer ADAER framework. Italicized names reflect the names of software packages in the Rust Package Registry. Dashed lines indicate future work for as yet unreleased cameras. Since multimodal sources can be transcoded to a single raw representation, we can have a simple source-modeled compression scheme. The representation supports bespoke event-based applications, while being backwards compatible with classical applications.

raw ADAER event stream. An ADAER stream may be lossy compressed with our bespoke source-modeled scheme. We further provide applications which interface with our decoder utility to provide functionality such as command-line metadata inspection, framed reconstruction, playback and visualization, and feature detection. Fig. 2 illustrates how these software components are connected.

Although the ADAER software suite provides several libraries for creating and processing our event streams, our GUI program *adder-viz* provides the most accessible starting point for researchers. Originally developed in late 2022, it provides researchers with a simple interface to experiment with various settings for ADAER transcoding. A user can simply drag and drop a video file into the window to begin a transcode operation. The program visualizes the transcode by continuously updating an image display with the most

recently fired event for each pixel. Transcoder settings are exposed with sliders and buttons, and the user can immediately observe the effect of changing each setting. *adder-viz* further provides an interface for replaying saved .adder files.

Our software is available on GitHub under the MIT license and distributed on the Rust Package Registry, crates.io. In our GitHub README, we provide user-friendly installation instructions, background reading, and a VirtualBox image. All source code documentation is available online on docs.rs. According to metrics on crates.io, the various ADAER packages have been collectively downloaded more than 150,000 times.

4 Software Enhancements

In the following, we describe in detail the major improvements made to our ADAER software since our last major version in January 2024, as summarized in ???. Many of these improvements are brought directly to the *adder-viz* program, while some are made to the underlying codec library.

4.1 Usability

Previously, we used the Bevy game engine [1] as our graphics back-end alongside the *egui* front-end library. We found that the entity component system (ECS) of Bevy was overly complex for our purposes, and the numerous dependencies caused extremely slow compile times. We thus replaced Bevy with the lightweight *eframe* renderer [6], which allows developers to compile their changes in a matter of seconds. A positive side effect of this library change is that our new compiled binary is 75% smaller than before.

Based on user feedback, we significantly streamlined the setup and usage of our program. Our prior version required the user to use the “nightly” branch of Rust if they wanted to perform source-modeled compression. This was a source of frustration, because the inherently unstable Rust branch would often break the program compilation. To solve this, we upstreamed improvements to the *arithmetic-coding* library [5] (which our codec depends on) that allow it to compile on the stable Rust branch. To make the program more approachable to new users, we also added a hover overlay to each transcoder setting. These overlays briefly describe what the settings do, and they give users the necessary context to search for further information in our documentation or associated papers. Finally, we added clearer instructions in our README for users to set up and install *adder-viz* for the first time. These efforts help ensure that our software is accessible for the open-source community.

4.2 Accuracy

Through our own experiments with visualizing ADAER transcode operations in *adder-viz* we found a number of latent bugs that were dramatically impacting the accuracy of our produced representations.

Firstly, we found that our transcoder did not output the correct data necessary to reconstruct a pixel value with zero intensity. Since our minimum D is 0, we do not fire an event until accumulating $2^0 = 1$ intensity unit. If a pixel has zero intensity for a sustained period, then, it will not produce the necessary events. To solve this, we defined a special case with a D constant for such intensities. Secondly, we corrected an algorithmic mistake in our DAVIS

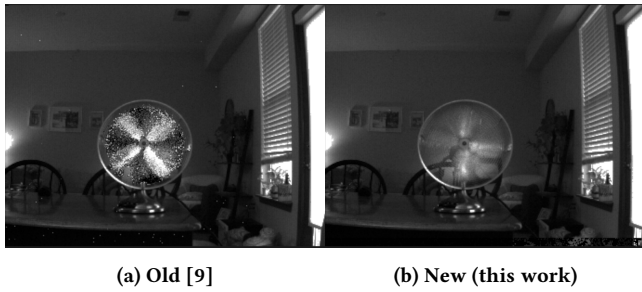


Figure 3: Qualitative example of the accuracy improvement to our DAVIS transcoder. Our events now have a more natural appearance. These images were recorded from reconstructed frames displayed in our *adder-viz* event video player.

transcoder. With our correction, the output ADAER events have higher accuracy and better temporal coherence. We illustrate this difference in Fig. 3.

4.3 Interactive Applications

We extended our FAST feature detection [18] application by implementing the Density-Based Spatial Clustering of Applications with Noise (DBSCAN) [7] algorithm to cluster the FAST features. We perform this clustering during ADAER transcode, operating at regular intervals (e.g., 30 times per second for event-based source, or matching the input rate of framed sources). These feature clusters then serve as prototypical regions for moving object detection, without the use of a learning-based method. We tuned the feature detection settings to avoid noisy regions and small clusters, instead prioritizing large clusters with several features. We provide a toggle to visualize these cluster bounding boxes on the live display in *adder-viz*, as shown in Fig. 1.

We also added a tool for a user to manually add a region of interest (ROI) during a transcode operation. The user simply clicks and drags within the *adder-viz* display to create an ROI. Pixels within the ROI will be set to have a high sensitivity, while pixels outside the ROI proceed as normal. By creating an ROI in combination with a high CRF value, the user can visualize the unique rate properties of ADAER: pixels with high sensitivity will produce more frequent events and a higher visual quality, and pixels with low sensitivity will produce few events. We illustrate this saliency-based rate control in Fig. 1.

4.4 Speed

Previously, our UI updates were performed on the same thread as the main transcoder process. Although convenient, the shortcoming of this design was severe lag when the transcoder could not process the incoming data in real time (e.g., when transcoding high-resolution video). Now, the UI is handled in a dedicated thread, and the transcoder uses a concurrent data structure to push updated visualization frames to the UI.

Additionally, our source-modeled compression mechanism frequently proved to be a bottleneck: when a group of events was compressed, the transcode and UI processes would freeze. We therefore moved the compressor to its own concurrent process, where

Res.	Raw events		Lossy compression	
	Grayscale	Color	Grayscale	Color
[9]	SD	60	60	52
	HD	53	39	14
	FHD	35	17	4
New	SD	150 (+150%)	129 (+115%)	63 (+21%)
	HD	85 (+60%)	62 (+59%)	32 (+128%)
	FHD	49 (+40%)	38 (+124%)	15 (+275%)

Table 1: Average frames per second at which the ToS framed video sequence can be transcoded within *adder-viz*. A CRF value of 3 (the default) was used for these experiments.

it receives events through message passing from the transcoder and compresses them asynchronously. Due to the computational complexity of the compression, we still experience slower performance when compression is enabled; however, the slowdown is now consistent across time, and does not cause noticeable pauses during transcoding.

We compared the new version of *adder-viz* to that of our prior work [9]. For input, we used the Tears of Steel (Tos) [17] sequence at various resolutions, both in grayscale and color, and both with and without our bespoke compression. The ToS sequence is a live-action movie with computer-generated video effects. We further validated the speed of transcoding and compressing full-motion DAVIS event camera video captured from a moving car. Our system has an Intel Core i9-14900K with 64 GB of RAM, running Ubuntu 24.04. We summarize our results in Tab. 1.

In total, our optimizations yield a tremendous improvement in the transcoder visualization speed. We see that the maximum framed transcoder speed we encountered before was merely 60 FPS, matching the refresh rate of our display. With the transcoder process divorced from the rendering thread, we now see much higher frame rates at low resolutions. Even in cases where we were not limited by the refresh rate or compressor lag, we still see improvements of 26% or more. In the best case, we observe an improvement of 275% when transcoding and compressing color Full-HD video. When transcoding our DAVIS video with frame-event fusion (mode (ii) of Fig. 2), we observe a maximum latency of 29 ms when compression is enabled. For transcoding only the DVS events (mode (iii) of Fig. 2), we observe a maximum latency of 25 ms with compression enabled. Finally, we found that the feature detection, clustering, and bounding box display systems had no measurable impact on performance with up to 25 bounding boxes displayed at once.

5 Conclusion

ADAER presents a unique alternative to traditional frame- and event-based sensing: it nonuniformly encodes intensity samples based on their rate of change, and the raw bitrate adjusts dynamically to the motion content. By increasing the temporal sparsity of the video representation, we can highly compress temporally stable regions without degrading the image quality. Our open-source software lowers barriers for new researchers to investigate event video. With further development of the compression and application techniques, we may see dramatic efficiency gains for large-scale video analysis systems.

Acknowledgments

Some text and images throughout were adapted from the first author's recent dissertation [8].

References

- [1] 2025. bevyengine/bevy. <https://github.com/bevyengine/bevy> original-date: 2020-01-18T21:13:55Z.
- [2] R. Wes Baldwin, Ruixu Liu, Mohammed Almatrafi, Vijayan Asari, and Keigo Hirakawa. 2023. Time-Ordered Recent Event (TORE) Volumes for Event Cameras. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 2 (Feb. 2023), 2519–2532. doi:10.1109/TPAMI.2022.3172212
- [3] Christian Brandli, Raphael Berner, Minhao Yang, Shih-Chii Liu, and Tobi Delbruck. 2014. A 240 × 180 130 dB 3 μs Latency Global Shutter Spatiotemporal Vision Sensor. *IEEE Journal of Solid-State Circuits* 49, 10 (Oct. 2014), 2333–2341. doi:10.1109/JSSC.2014.2342715
- [4] Christian Brandli, Lorenz Muller, and Tobi Delbruck. 2014. Real-time, high-speed video decompression using a frame- and event-based DAVIS sensor. In *2014 IEEE International Symposium on Circuits and Systems (ISCAS)*. 686–689. doi:10.1109/ISCAS.2014.6865228 ISSN: 2158-1525.
- [5] danielleades. 2025. danielleades/arithmetic-coding. <https://github.com/danielleades/arithmetic-coding> original-date: 2022-03-16T17:20:54Z.
- [6] Emil Ernerfeldt. 2025. emilk/egui. <https://github.com/emilk/egui> original-date: 2019-01-13T15:39:15Z.
- [7] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD'96)*. AAAI Press, Portland, Oregon, 226–231.
- [8] Andrew Freeman. 2024. *Rethinking Video with a Universal Event-Based Representation*. Ph. D. Dissertation. The University of North Carolina at Chapel Hill University Libraries. doi:10.17615/5BSV-BZ25
- [9] Andrew C. Freeman. 2024. An Open Software Suite for Event-Based Video. In *Proceedings of the ACM Multimedia Systems Conference 2024*. ACM, Bari Italy, 271–277. doi:10.1145/3625468.3652169
- [10] Andrew C. Freeman, Ketan Mayer-Patel, and Montek Singh. 2024. Accelerated Event-Based Feature Detection and Compression for Surveillance Video Systems. In *Proceedings of the 15th ACM Multimedia Systems Conference (MMSys '24)*. Association for Computing Machinery, New York, NY, USA, 132–143. doi:10.1145/3625468.3647618
- [11] Andrew C. Freeman, Montek Singh, and Ketan Mayer-Patel. 2023. An Asynchronous Intensity Representation for Framed and Event Video Sources. In *Proceedings of the 14th ACM Multimedia Systems Conference*. ACM, Vancouver BC Canada, 74–85. doi:10.1145/3587819.3590969
- [12] Guillermo Gallego, Tobi Delbrück, Garrick Orchard, Chiara Bartolozzi, Brian Taba, Andrea Censi, Stefan Leutenegger, Andrew J. Davison, Jörg Conradt, Kostas Daniilidis, and Davide Scaramuzza. 2022. Event-Based Vision: A Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 01 (Jan. 2022), 154–180. doi:10.1109/TPAMI.2020.3008413 Publisher: IEEE Computer Society.
- [13] Daniel Gehrig, Antonio Loquercio, Konstantinos Derpanis, and Davide Scaramuzza. 2019. End-to-End Learning of Representations for Asynchronous Event-Based Data. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, Seoul, Korea (South), 5632–5642. doi:10.1109/ICCV.2019.00573
- [14] Patrick Lichtsteiner, Christoph Posch, and Tobi Delbruck. 2008. A 128x128 120 dB 15 microsecond Latency Asynchronous Temporal Contrast Vision Sensor. *IEEE Journal of Solid-State Circuits* 43, 2 (Feb. 2008), 566–576. doi:10.1109/JSSC.2007.914337 Conference Name: IEEE Journal of Solid-State Circuits.
- [15] Ana I. Maqueda, Antonio Loquercio, Guillermo Gallego, Narciso Garcia, and Davide Scaramuzza. 2018. Event-Based Vision Meets Deep Learning on Steering Prediction for Self-Driving Cars. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Salt Lake City, UT, 5419–5427. doi:10.1109/CVPR.2018.00568
- [16] Henri Rebecq, Timo Horstschaefer, and Davide Scaramuzza. 2017. Real-time Visual-Inertial Odometry for Event Cameras using Keyframe-based Nonlinear Optimization. In *Proceedings of the British Machine Vision Conference 2017*. British Machine Vision Association, London, UK, 16. doi:10.5244/C.31.16
- [17] Ton Roosendaal. 2012. About | Tears of Steel. <https://mango.blender.org/about/>
- [18] E. Rosten and T. Drummond. 2005. Fusing points and lines for high performance tracking. In *Tenth IEEE International Conference on Computer Vision (ICCV'05) Volume 1*, Vol. 2. 1508–1515 Vol. 2. doi:10.1109/ICCV.2005.104 ISSN: 2380-7504.
- [19] Alex Zihao Zhu, Liangzhe Yuan, Kenneth Chaney, and Kostas Daniilidis. 2018. EV-FlowNet: Self-Supervised Optical Flow Estimation for Event-based Cameras. In *Robotics: Science and Systems XIV*. doi:10.15607/RSS.2018.XIV.062 arXiv:1802.06898 [cs].